DEFINITE ITERATION

This is used when the number of iterations, or turns of the loop, is known in advance.

In the Pearson Edexcel pseudocode there are two ways of doing this using `REPEAT…END REPEAT` and `FOR…END FOR`.

An example of a `REPEAT` loop is shown below.

```
REPEAT 50 TIMES
  SEND '*' TO DISPLAY
END REPEAT
```

Using a `FOR` loop, this would be:

```
FOR times FROM 1 TO 100 DO
    SEND '*' TO DISPLAY
END FOR
```

`FOR` loops can also include a step so that the counting is not consecutive. A step is included in the following pseudocode example.

```
FOR times FROM 0 TO 100 STEP 25 DO
    SEND times TO DISPLAY
END FOR
```

The output would be 0, 25, 50, 75.

USE	EXAMPLE	RESULT
Using the 'range' command	`for x in range(6):` `  print(x)`	0 1 2 3 4 5
Stipulating a start number so that it does not begin at 0	`for x in range(2,6):` `  print(x)`	2 3 4 5

Using a step by adding a third number into the brackets	`for x in range(0,100,25):`
	`  print(x)`
	To also print 100 you would use:
	`for x in range(0,101,25):`
		0
		25
		50
		75

Printing items from a list	`aList = ['red', 'blue', 'green', 'yellow', 'purple', 'orange']`
	`for colour in aList:`
	`  print (colour)`
	The number of iterations does not have to be given as the length of the list is used.
		red
		blue
		green
		yellow
		purple
		orange

(continued)

(continued)

USE	EXAMPLE	RESULT

Leaving out items from the list	`aList = ['red', 'blue', 'green', 'yellow', 'purple', 'orange']`
	`for colour in aList:`
	`  if colour == 'yellow':`
	`  continue`
	`  print (colour)`
		red
		blue
		green
		purple
		orange

▲ Table 2.2 Python

USE	EXAMPLE	RESULT

Using a `for` loop	`for(int x = 0; x < 6; x++) {`
	`  System.out.println(x);`
	`}`
		0
		1
		2
		3
		4
		5

Stipulating a start number so that it does not begin at 0	`for(int x = 2; x < 6; x++) {`
	`  System.out.println(x);`
	`}`
		2
		3
		4
		5

Using a step by adding an amount to increase by	`for(int x = 0; x < 100; x+=25) {`
	`  System.out.println(x);`
	`}`
	To also print 100 you would use:
	`for(int x = 0; x <= 100; x+=25) {`
	`  System.out.println(x);`
	`}`
		0
		25
		50
		75

Printing items from a list	`String[] aList = {"red", "blue", "green", "yellow", "purple", "orange"};`
	`for(String colour: aList) {`
	`  System.out.println(colour);`
	`}`
		red
		blue
		green
		yellow
		purple
		orange

Leaving out items from the list	`String[] aList = {"red", "blue", "green", "yellow", "purple", "orange"};`
	`for(String colour: aList) {`
	`  if(colour == "yellow") {`
	`  continue;`
	`  }`
	`  System.out.println(colour);`
	`}`
		red
		blue
		green
		purple
		orange

▲ Table 2.3 Java

SUBJECT VOCABULARY

infinite loop a loop that is never-ending since the condition required to **terminate** the loop is never reached

GENERAL VOCABULARY

terminate to cause something to end or stop

example of an '**infinite loop**'. The algorithm should have been written as shown below.

```
SET index TO 1
WHILE index < 10
      SEND index TO DISPLAY
      SET index TO index + 1
END WHILE
```

SKILLS REASONING, PROBLEM SOLVING

ACTIVITY 29

FINDING AND CORRECTING ERRORS

Find and correct the errors in these algorithms.

Example 1

```
SET index TO 1
WHILE index < 10
    SEND index TO DISPLAY
END WHILE
```

Example 2

```
SET index TO 1
WHILE index < 10
    SEND index TO DISPLAY
    SET index TO index - 1
END WHILE
```

Example 3

```
SET index TO 1
WHILE index < 1
    SEND index TO DISPLAY
    SET index TO index + 1
END WHILE
```

TRACE TABLES

The formal way of checking the logic of an algorithm is to use a **trace table**.

SUBJECT VOCABULARY

logic error an error in an algorithm that results in incorrect or unexpected behaviour

trace table a technique used to identify any logic errors in algorithms. Each column represents a variable or output and each row a value of that variable

WORKED EXAMPLE

Create a trace table for the following algorithm written in Pearson Edexcel pseudocode.

```
SET number TO 3
FOR index FROM 1 TO 5 DO
    SET number1 TO number * index
    SET number2 TO number1 * 2
    IF number2 > 20 THEN
        SEND number2 TO DISPLAY
    END IF
END FOR
```

This algorithm can be traced as in Table 2.12.

NUMBER	INDEX	NUMBER1	NUMBER2	OUTPUT
3				
3	1	3	6	
3	2	6	12	
3	3	9	18	
3	4	12	24	24
3	5	15	30	30

▲ **Table 2.12** Trace table

The value of the variable number remains at 3 throughout, but as the index increases from 1 to 5, then so do the values of `numberl` and `number2`.

When the value of `number2` is greater than 20, its value is output.

WORKED EXAMPLE

Here is a program.

Python

```
y = 2
for x in range(1, 7):
      y = y + x
print(y)
```

Java

```
class Main {
  public static void main(String[] args) {
    int y = 2;
    for(int x = 1; x < 7; x++) {
      y += x;
    }
    System.out.println(y);
  }
}
```

C#

```
int number = 3;
int number1, number2;

for (int index = 1; index <= 5; index++)
{
    number1 = number * index;
    number2 = number1 * 2;
    if (number2 > 20)
    {
        Console.WriteLine(number2);
    }
}
```

X	Y	OUTPUT	EXPLANATION
1	2		When the loop starts, X becomes 1 and Y already is equal to 2.
2	3		When X is incremented to 2, Y is equal to 3 (2+1) from the previous loop.
3	5		When X is incremented to 3, Y= 3 + 2 from the previous loop.
4	8		Explanations as above.
5	12		
6	17		
6	23	23	The final value of Y is output.

▲ **Table 2.13** Trace table of the programs above

ACTIVITY 30

Complete a trace table for this program written in a high-level language.

Python

```
number1 = 2
number2 = 3
for index in range(1, 6)
      number1 = number1 * index
      number2 = number2 + number1
```